

Cartographie sémantique FSBM

Elbah Nevissa

1. Contexte

Ce projet vise à construire un pipeline de données pour analyser la production scientifique des enseignants-chercheurs de la Faculté des Sciences Ben M'Sick. Les données proviennent de Google Scholar et sont structurées afin de permettre une recherche sémantique sur les abstracts des publications.

2. Architecture du pipeline

Etape 1 - Scraping

Le script `scraper.py` lit `input_faculty_list.json`, récupère les profils Google Scholar avec `scholarly`, extrait les métriques des chercheurs et les publications, puis sauvegarde progressivement les données dans `raw_scholar_data.json`.

Robustesse ajoutée :

- reprise depuis le fichier déjà scrapé ;
- sauvegarde après chaque profil ;
- pauses aléatoires entre requêtes ;
- gestion des erreurs par profil et par publication ;
- extraction de `pdf_url` lorsque disponible.

Etape 2 - Nettoyage

Le script `cleaner.py` nettoie les abstracts, supprime les articles sans résumé exploitable, normalise les dates et produit :

- `processed_abstracts.json` pour garder la structure chercheur -> articles ;
- `processed_abstracts.csv` pour l'analyse tabulaire ;
- `processed_abstracts.parquet` si `pyarrow` ou `fastparquet` est installé.

Etape 3 - Embeddings et recherche sémantique

Le script `generate_vectors_and_index.py` encode les abstracts nettoyés. Il tente d'abord d'utiliser le modèle `zeroentropy/zembed-1-embedding`. Si l'environnement ne permet pas de télécharger ou charger le modèle, il génère un fallback déterministe local afin de garder une démonstration exécutable.

Sorties :

- `fsbm_final_dataset_with_embeddings.json` avec le champ `embedding_zembed1` ;
- `semantic_index.npz` contenant la matrice d'embeddings normalisée ;
- `embedding_metadata.json` décrivant le modèle utilisé.

Le script `semantic_search.py` calcule la similarité cosinus entre une requête utilisateur et les publications indexées.

3. Dataset obtenu

L'extraction incluse contient un échantillon représentatif de chercheurs FSBM. Après nettoyage, le dataset conserve les articles ayant des abstracts exploitables et les champs suivants :

- identifiant chercheur ;
- nom complet ;
- affiliation ;
- métriques bibliométriques ;
- titre, auteurs, année, journal, citations ;
- abstract brut ;
- abstract nettoyé ;
- embedding `embedding_zembed1`.

4. Exemple d'utilisation

```
python cleaner.py
python generate_vectors_and_index.py
python semantic_search.py "deep learning for medical imaging" --top-k 5
```

Pour une démo hors-ligne :

```
python generate_vectors_and_index.py --offline
python semantic_search.py "machine learning cancer diagnosis" --top-k 5
```

5. Conclusion

Le projet couvre les livrables demandés : scraping, nettoyage, dataset final structure, vectorisation des abstracts et démonstration de recherche sémantique. La version finale est reproductible et documente clairement le cas où le modèle `zembed-1` n'est pas disponible localement.